# MATERIAL EXTRA

## Servidor NodeJS com Express

growdev

# APRESENTAÇÃO

Ao começar a trabalhar com desenvolvimento backend em Node.js, uma das ferramentas mais populares é o **Express**, um framework minimalista e altamente flexível que facilita a criação de servidores e APIs. Neste guia, vamos explorar os principais conceitos relacionados à configuração e uso de um servidor Express, abordando suas funcionalidades de forma detalhada e prática.

# INTRODUÇÃO AO EXPRESS

O Express é um framework que abstrai as complexidades de trabalhar diretamente com o módulo HTTP nativo do Node.js, fornecendo uma maneira mais simples e organizada de gerenciar rotas, lidar com requisições, configurar middlewares e processar respostas.

### Por que o Express é útil?

- Ele reduz a quantidade de código necessário para criar um servidor. Oferece uma maneira eficiente de lidar com diferentes métodos HTTP (GET, POST, PUT, etc.).
- Possui uma comunidade ativa, com muitos pacotes e middlewares disponíveis para integração com bancos de dados, autenticação, validação e muito mais.
- Um servidor básico com Express pode ser configurado em poucas linhas de código, tornando-o ideal tanto para iniciantes quanto para desenvolvedores experientes.

# INICIALIZAÇÃO DO PROJETO NODEJS

1. `npm init -y`
   inicializa um projeto node

2. `npm install express`
   Adicionar a lib express como dependência de produção

3. Editar o arquivo **package.json** para conter a configuração de módulo.

```
{
  // ...
  "type": "module",
  // ...
}
```

4. Criar o diretório **src** e o arquivo **index.js** dentro dele.

5. Escrever o seguinte código no arquivo **index.js**

```
import express from "express";

const app = express();
app.use(express.json());

app.get("/hello", (req, res) => {
  res.send("Hello, Express!");
});

app.listen(3000, () => {
  console.log("Servidor rodando na porta 3000");
});
```

Essa configuração básica já permite que o servidor responda a requisições na rota **`/hello`** e funcione na porta **3000**.
Para rodar o servidor execute o comando **`node src/index.js`**

# VARIÁVEIS DE AMBIENTE

Variáveis de ambiente são usadas para armazenar configurações sensíveis, como a porta do servidor ou credenciais, sem que esses valores sejam diretamente codificados no arquivo principal. Para gerenciá-las, usamos o pacote dotenv.

### Passo a passo para usar dotenv:

1. Instale o pacote.
   **`npm install dotenv`**

2. Crie um arquivo **.env** na raiz do projeto e adicione as variáveis.
   **`PORT=3000`**

3. No código, importe e configure o dotenv.

```
// ...
import * as dotenv from "dotenv";

dotenv.config();

// ...

const porta = process.env.PORT;
app.listen(porta, () => {
  console.log(`Servidor rodando na porta ${porta}`);
});

// ...
```

Isso torna o código mais seguro e flexível, especialmente em ambientes de produção.

## NODEMON PARA DESENVOLVIMENTO

Durante o desenvolvimento, reiniciar manualmente o servidor toda vez que um arquivo é alterado pode ser tedioso. O Nodemon resolve esse problema ao monitorar as alterações nos arquivos e reiniciar automaticamente o servidor.

### Configuração do Nodemon:

1. Instale como dependência de desenvolvimento:
   **`npm install -D nodemon`**

2. Configure um script no arquivo package.json:
   ```
   "scripts": {
      "dev": "nodemon src/index.js"
   }
   ```

3. Execute o servidor em modo de desenvolvimento:
   **`npm run dev`**

Agora, sempre que você modificar um arquivo, o Nodemon reiniciará o servidor automaticamente.

## DECLARAÇÃO DE ROTAS DA API

O Express permite lidar com os métodos HTTP mais comuns: GET, POST, PUT, PATCH e DELETE, essenciais para criar uma API REST. Cada método tem um propósito específico:

- **GET**: Busca informações.
- **POST**: Cria um recurso.
- **PUT**: Atualiza um recurso inteiro.
- **PATCH**: Atualiza parcialmente um recurso.
- **DELETE**: Remove um recurso.

Exemplos de Rotas:

```
// GET: Recupera informações
app.get("/users", (req, res) => {
  res.send("Listando todos os usuários");
});

// POST: Cria novos recursos
app.post("/users", (req, res) => {
  res.send("Usuário criado");
});

// PUT: Atualiza um recurso inteiro
app.put("/users/:id", (req, res) => {
  res.send(`Usuário ${req.params.id} atualizado`);
});

// PATCH: Atualiza parte de um recurso
app.patch("/users/:id", (req, res) => {
  res.send(`Informações do usuário ${req.params.id} atualizadas`);
});

// DELETE: Remove um recurso
app.delete("/users/:id", (req, res) => {
  res.send(`Usuário ${req.params.id} removido`);
});
```

# REQUEST E RESPONSE

No Express, as requisições são representadas pelo objeto **req**, enquanto as respostas são manipuladas pelo objeto **res**.

- **req**: Contém informações enviadas pelo cliente, como cabeçalhos, parâmetros, e o corpo da requisição.
- **res**: Permite enviar respostas ao cliente, definir status HTTP, enviar mensagens ou arquivos (JSON, XML, HTML, etc).

```
app.get("/example", (req, res) => {
  console.log(req.ip);
  console.log(req.host);
  console.log(req.body);
  console.log(req.query);
  console.log(req.params);
  console.log(req.headers);

  res.status(200).send({
    message: "Requisição bem-sucedida!",
  });
});
```

## BODY

O corpo da requisição (body) é geralmente usado em métodos como **POST**, **PUT** e **PATCH**, para enviar dados estruturados em formato JSON.

```
app.post("/users", (req, res) => {
  const userData = req.body; // Dados enviados no body

  res.status(201).send({
    message: "Usuário criado",
    user: userData
  });
});
```

## QUERY PARAMS

Os parâmetros de consulta (query params) são enviados na URL após o símbolo **?**, separados pelo símbolo **&** e geralmente são usados para filtros ou paginação. Eles são acessados através de **`req.query`**.

```
app.get("/users", (req, res) => {
  const { name, age } = req.query; // Exemplo: /users?name=João&age=30

  res.status(200).send({
    filters: {
      name,
      age,
    },
  });
});
```

## PATH PARAMS

Os parâmetros de rota (path params) fazem parte do caminho da URL e são usados para identificar recursos específicos. São acessados através de **`req.params`**.

```
app.get("/users/:id", (req, res) => {
  const { id } = req.params; // Exemplo: /users/123

  res.status(200).send({
    message: `Usuário com ID ${id}`,
  });
});
```

## CONCLUSÃO

Com o Express, criar servidores e APIs se torna uma tarefa acessível e eficiente. Ele oferece ferramentas para lidar com rotas, métodos HTTP e dados de requisições de forma prática, além de permitir a integração ferramentas como dotenv e Nodemon, que tornam o desenvolvimento mais organizado e produtivo.

# BONS ESTUDOS

growdev